JavaScript Notes

Variables · Data Types · Operators · Type Coercion

01 Variables — var vs let vs const

var — Purana	let — Badalne wali	const — Default choice
Function scope	Block scope	Block scope
Hoist hota hai — bugs deta hai	Loop counters, re-assign values ke liye	Jab value change na ho — mostly yahi use karo
`var x = 1 x = 2 // ok`	`let x = 1 x = 2 // ok`	`const x = 1 x = 2 // ERROR`

Example code

```
const name = "Rahul"; // string — fix hai
let score = 0; // number — baad mein badlega
const PI = 3.14; // number — fix hai
const isLoggedIn = true; // boolean

score = 10; // ok — let re-assign ho sakta hai
// name = "Amit"; // ERROR — const re-assign nahi hota
```

02 Data Types — 7 primitive types

String

```
const a = "Hello"
const b = 'World'
const c = `Hi ${a}`
a.length // 5
```

Number

```
const age = 25
const pi = 3.14
NaN + 1 // NaN
typeof age // number
```

Boolean

```
const ok = true
const no = false
5 > 3 // true
if(ok) { ... }
```

Null

```
let user = null
// jaanboojhkar khaali
typeof null // "object"
// (JS ka purana bug!)
```

Undefined

```
let x
console.log(x) // undefined
const o = {}
o.name // undefined
```

Object / Array

```
const p = { name: "Ali" }
p.name // "Ali"
const a = [1,2,3]
a[0] // 1
```

03 Operators

Arithmetic + - * / % **

```
const a = 10, b = 3
a + b // 13 — jod a - b // 7 — ghataav
a * b // 30 — gunaa a / b // 3.33 — bhaag
a % b // 1 — remainder a ** b // 1000 — power
```

Comparison == === != !== > < >= <=

```
5 == "5" // true — sirf value (AVOID!)
5 === "5" // false — value + type (HAMESHA yahi)
5 !== 3 // true
10 > 5 // true
```

Logical && || !

```
const age = 20, hasId = true
age >= 18 && hasId // true — dono true chahiye
age < 18 || hasId // true — koi ek true ho
!hasId // false — ulta kar deta hai
```

Assignment = += -= *= /= ++ --

```
let x = 10
x += 5 // x=15 x -= 3 // x=12
x *= 2 // x=24 x /= 4 // x=6
x++ // x=7 x-- // x=6
```

typeof

```
typeof "hello" // "string"
typeof 42 // "number"
typeof true // "boolean"
typeof null // "object" — JS bug!
typeof undefined // "undefined"
```

04 Type Coercion — JS ka dark magic

2 tarah ki type casting

Implicit Coercion	Explicit Conversion
JS khud type badalta hai — tum kuch nahi karte. Yahi bugs ka source hai.	Tum khud convert karte ho — Number(), String(), Boolean() se. Safe aur predictable.
`"3" + 1 // "31" (shock!)`	`Number("3") + 1 // 4`

Rule 1 — + operator: number ya string?

Agar koi bhi operand string hai → concat. Dono number → addition.

```
1 + 2 // 3 — dono number, addition
"1" + 2 // "12" — string hai, concat!
1 + "2" // "12" — same
1 + 2 + "3" // "33" — 1+2=3, phir 3+"3"="33"
"1" + 2 + 3 // "123" — "1"+2="12", "12"+3="123"
```

Rule 2 — -, *, /, % — hamesha number banata hai

Yeh operators string ko number mein convert karne ki koshish karte hain.

```
"10" - 3 // 7 — "10" → 10, phir minus
"5" * "3" // 15 — dono convert
"abc" - 1 // NaN — "abc" valid number nahi
"6" / "2" // 3 — works!
```

Rule 3 — Boolean ki numeric value

true = 1 | false = 0 — math mein yahi hoga.

```
true + 1 // 2 — true → 1
false + 1 // 1 — false → 0
true + true // 2 — 1+1
true + "1" // "true1" — string se milne par concat!
false * 100 // 0 — 0*100
```

Rule 4 — null aur undefined

null → 0 in math. undefined → NaN hamesha.

```
null + 1 // 1 — null → 0
null + null // 0 — 0+0
undefined + 1 // NaN — undefined → NaN
null + undefined // NaN — 0 + NaN = NaN
null + "hi" // "nullhi" — + string = concat!
```

Rule 5 — 6 Falsy values — yeh 6 hi hain

if() mein yeh 6 false maante hain. Baaki sab truthy hain.

```
false, 0, "", null, undefined, NaN // yeh 6 falsy hain

// Truthy surprises:
"0" // truthy — non-empty string
"false" // truthy — non-empty string
[] // truthy — empty array
{} // truthy — empty object
-1 // truthy — 0 ke alawa sab numbers
```

Tricky Questions — answers with explanation

Expression	Result	Kyun?
`true + 1`	2	true → 1. Phir 1 + 1 = 2.

`false + 1`	`1`	false → 0. Phir 0 + 1 = 1.
`null + undefined`	`NaN`	null → 0. undefined → NaN. 0 + NaN = NaN.
`null + 1`	`1`	null → 0. 0 + 1 = 1.
`"" + 0`	`"0"`	"" string hai → concat hoga. "" + "0" = "0".
`"3" - 1`	`2`	"3" → 3 (minus number chahta hai). 3 - 1 = 2.
`"3" + 1 - 1`	`30`	"3"+1 = "31". Phir "31"-1 = 30. Shocking!
`[] + []`	`""`	[] → empty string. "" + "" = "".
`[] + {}`	`"[object Object]"`	[] → "", {} → "[object Object]". Concat.
`false == "0"`	`true (!)`	false → 0, "0" → 0. 0==0 → true. Isliye === use karo!
`null == undefined`	`true`	Special case: yeh dono sirf aapas mein == hain.
`NaN === NaN`	`false (!)`	NaN khud se equal nahi hota. Use: Number.isNaN(x)

Explicit conversion — safe tarika

```
// String → Number
Number("42") // 42 Number("") // 0
Number("abc") // NaN Number(true) // 1
Number(null) // 0 Number(undefined) // NaN
parseInt("42px") // 42 — sirf number part parseFloat("3.5m") // 3.5

// Number → String
String(42) // "42" (42).toString() // "42"

// Boolean convert
Boolean(0) // false Boolean("hello") // true
Boolean(null) // false !!"text" // true (shorthand)
!!0 // false
```

Quick Reference — yaad rakhne wali baatein

Rule	Remember this
`===`	Hamesha triple equal use karo — type bhi check karta hai
`== is dangerous`	false == "0" → true hota hai! Isliye avoid karo
`+ with string`	Ek bhi string ho to concat hoga, addition nahi
`-, *, / strings`	Yeh operators string ko number banate hain — ya NaN dete hain
`null vs undefined`	null → 0 in math | undefined → hamesha NaN
`NaN check`	NaN === NaN → false! Use Number.isNaN(x)
`6 falsy values`	false, 0, "", null, undefined, NaN — bas yahi 6